Programmbeschreibung

Strategielogik

➢ Definition des Marktpreises

- Ein Marktpreis muss vom Käufer und Verkäufer akzeptiert werden.
- Zum Beispiel

  Ein Bier kostet 2 Euro und der Preis wird vom Käufer und Verkäufer akzeptiert. Deshalb ist der Preis sinnvoll.

  Wenn ein Bier dagegen 2.000 Euro kostet, können die meisten Verbraucher nicht diesen Preis akzeptieren. Deshalb ist dieser Preis sinnlos, weil die meisten Verbraucher diesen Preis zu hoch und irrational finden.
- Wenn der Zeitpunkt und die äußere Umgebung stabil sind, bestehen zwischen verschiedenen Produkten der Preisverhältnisse vom Preis am Markt. Außerdem ist die Preisverhältnisse stabil und wird nicht groß verändern.
- Wenn sich diese Preisverhältnisse groß verändern, können dafür mehrere Gründe verantwortlich sein:
  1. Äußere Einflüsse haben sich groß verändert.
  2. Das Produkt hat sich groß verändert.
  3. Am Markt gibt es große oder unnormale Handelsaktivitäten. Z.B. Fehlorder.

➢ Handelslogik

- Die Gewinnquelle liegt in der Korrektur und Stabilisierung der Preisverhältnisse am Markt.
- Zum Beispiel der Wechselkurs zwischen USD und Euro.
  1. Die Handelsrichtungen USD/EUR und EUR/USD werden gleichzeitig berücksichtigt und ausgeführt.
  2. Wenn das Preisverhältnisse innerhalb des statistischen Bereichs liegt und kein Ausreißer ist, werden Positionen eröffnet.
  3. Sobald das Gewinnziel oder die Verlustgrenze erreicht wird, werden die Positionen geschlossen und in die ursprüngliche Währung zurückgetauscht.
  4. Gewinne, die über dem festgelegten Gewinnziel liegen, werden als Reserve gespart, um verlustreiche Positionen auszugleichen.

Technische Umgebung

- **Programmiersprache**
  - Python 3.11.6
- **Python-Bibliotheken**
  - Standardbibliotheken :
    math, datetime, importlib , os, time, hmac, hashlib, socket, json, subprocess
  - Externe Python-Bibliotheken :
    Pandas, numpy, requests
  - Google API :
    google.oauth2.service_account, googleapiclient.discovery, googleapiclient.errors
- **Externe API**
  - MEXC_V3
- **Datenspeicherung**
  - Google Sheets
- **Verwendete Cloud-Plattform**
  - Google Cloud
  - Railway

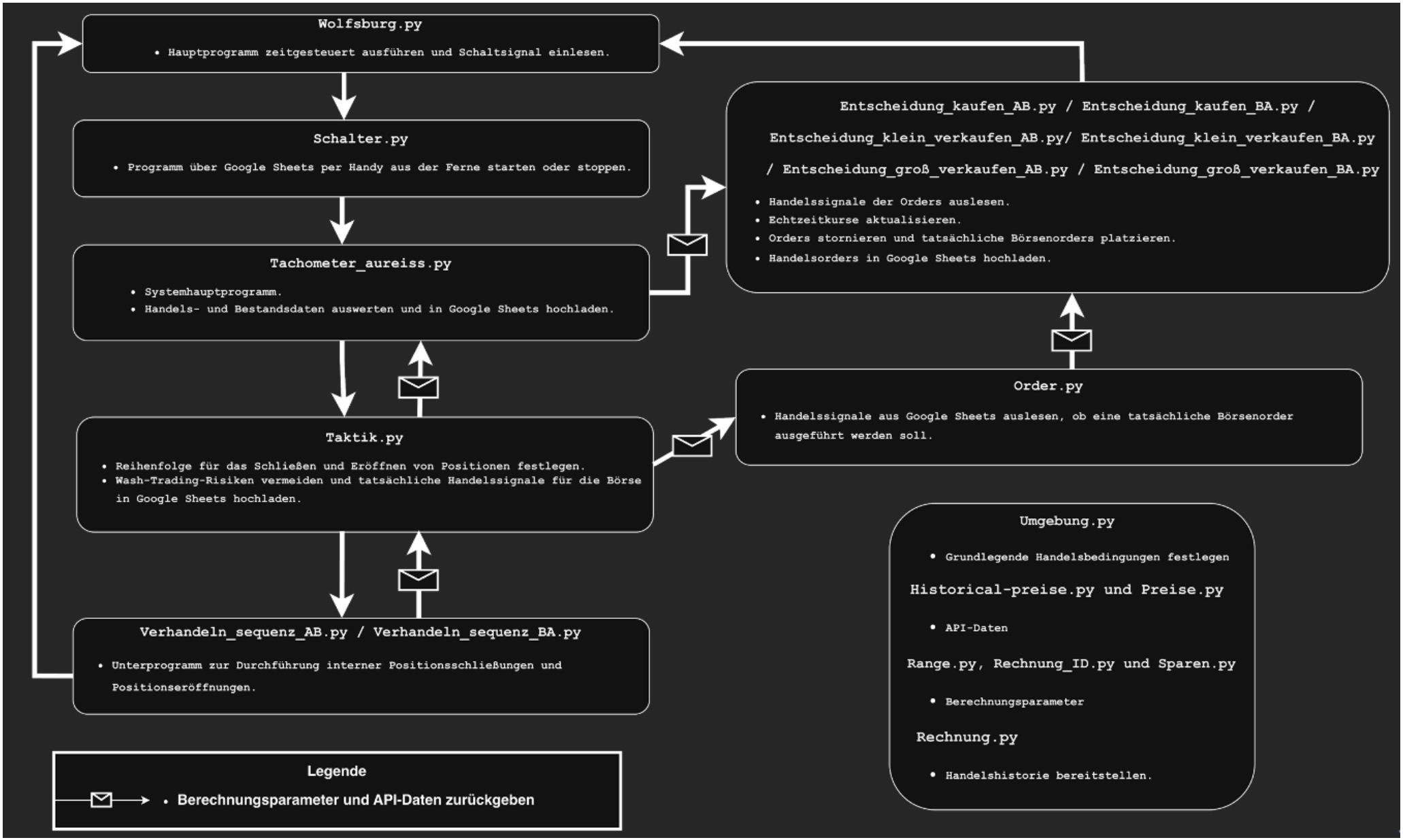
Unterprogrammfunktion

- **Wolfsburg.py:**
  - Hauptprogramm zeitgesteuert ausführen und Schaltsignal einlesen.
  - Ausführungsintervall des Hauptprogramms steuern.
- **Schalter.py:**
  - Schaltsignal aus Google Sheets einlesen.
  - Programm über Google Sheets per Handy aus der Ferne starten oder stoppen.

- **Tachometer_aureiss.py:**
  - Systemhauptprogramm.
  - Handels- und Bestandsdaten auswerten und in Google Sheets hochladen.
- **Taktik.py:**
  - Reihenfolge für das Schließen und Eröffnen von Positionen festlegen.
  - Wash-Trading-Risiken vermeiden und tatsächliche Handelssignale für die Börse in Google Sheets hochladen.
- **Verhandeln_sequenz_AB.py und Verhandeln_sequenz_BA.py:**
  - Unterprogramm zur Durchführung interner Positionsschließungen und Positionseröffnungen.
- **Kaufen_AB.py und Kaufen_BA.py:**
  - Bei erfüllten Handelsbedingungen eine interne Position eröffnen.
  - Handelsorders in Google Sheets hochladen.
- **Klein_verkaufen_AB.py und Klein_verkaufen_BA.py:**
  - Die Order mit den niedrigsten Eröffnungskosten ermitteln.
  - Bei erfüllten Gewinnmitnahme- oder Liquidationsbedingungen eine interne Position eröffnen.
  - Handelsorders in Google Sheets hochladen.
- **Groß_verkaudfen_AB.py und Groß_verkaudfen_BA.py:**
  - Die Order mit den höchsten Eröffnungskosten ermitteln.
  - Bei erfüllten Gewinnmitnahme- oder Liquidationsbedingungen eine interne Position eröffnen.
  - Handelsorders in Google Sheets hochladen.
- **Order.py:**
  - Handelssignale aus Google Sheets auslesen, ob eine tatsächliche Börsenorder ausgeführt werden soll.

- **Entscheidung_kaufen_AB.py, Entscheidung_kaufen_BA.py, Entscheidung_klein_verkaufen_AB.py, Entscheidung_klein_verkaufen_BA.py, Entscheidung_groß_verkaufen_AB.py und Entscheidung_groß_verkaufen_BA.py:**
  - Handelssignale der Orders auslesen.
  - Echtzeitkurse aktualisieren.
  - Orders stornieren und tatsächliche Börsenorders platzieren.
  - Handelsorders in Google Sheets hochladen.
- **Umgebung.py:**
  - Grundlegende Handelsbedingungen festlegen.
  - Positionseröffnung, Handelsvolumen, Eröffnungsdatum, Handelsinstrument usw.
- **Historical-preise.py:**
  - Historische Kursdaten über die Börsen-API abrufen.
- **Preise.py:**
  - Marktpreise über die Börsen-API abrufen.
- **Range.py:**
  - Statistische Intervalle anhand historischer Kursdaten berechnen.
- **Rechnung.py:**
  - Handelshistorie abrufen.
- **Rechnung_ID.py:**
  - Neue Order-ID generieren.
- **Sparen.py:**
  - Rücklage für Überschussgewinne berechnen.

Systemarchitektur



Geplante Verbesserungen

- **Umstellung der Speicherung von Handelsdaten auf PostgreSQL**

  Die Speicherung der Handelsdaten soll zukünftig von Google Sheets auf PostgreSQL umgestellt werden.

  Dadurch soll die Anzahl der Zugriffe auf die Google Sheets API reduziert werden. Außerdem soll die Datenverarbeitung effizienter werden und die Ausführungszeit des Programms verkürzt werden.

- **Optimierung der Berechnungen in Range.py**

  Die Berechnungsergebnisse sollen vorübergehend in einer Datenbank gespeichert werden.

  Wenn die aktuellen historischen Kursdaten mit den Daten übereinstimmen, die für die gespeicherten Berechnungsergebnisse verwendet wurden, soll die Berechnung nicht erneut durchgeführt werden.

  Dadurch können unnötige Neuberechnungen vermieden, Rechenressourcen eingespart und die Ausführungszeit des Programms verkürzt werden.